

QOMM

Oblivious Market Making

構成・証明・実測

$n=7$, $T=2$, Shamir

Pedersen / ed25519

全数値は artifacts/*.json

コミットしたものと、 回路に入れたものを、 同じだと誰が言うのか

公開検証可能 MPC の継ぎ目に、証明を書いて初めて見つかった健全性の欠陥があった。その話と、直した後の値段。

2026-08

このスライドが引き受ける範囲

デッキ	読者	扱うもの
qomm_intro qomm_slides qomm_tech (本編)	予備知識なし 分野を知っている 暗号の術語を説明せず使っ てよい	なぜ値段を聞くこと自体が損なのか。数式なし 全体像・費用・制度上の位置・先行研究との差 構成そのもの、健全性の証明、実測、そして外した 予測

この編の主題は一点である。公開検証可能 MPC は「計算が正しい」ことを外から確かめられるようにするが、ノードが回路に入れた値が、配られた値と同じかは、その外側にある。

ここを塞ぐ検査を作り、証明を書く過程で健全性の欠陥を見つけ、直し、値段を測った — 1 ラウンド、通信 +0.39%。

§1

設定

$n = 7$ 、 $T = 2$ 、そして「整数上で分割する」という選択

パラメータと敵

参加者は二種類、交わらない

入力者 $\mathcal{I}$ ：依頼ごとのテイカー 1、方針ごとのメイカー。値を分割して渡すだけで、計算はしない。

計算者 $\mathcal{P} = \{P_1, \dots, P_7\}$ ：すべての値のシェアを 1 つずつ持ち、**どの値も丸ごととは持たない**。

敵

静的。計算者を高々 $T = 2$ 、入力者は何人でも。

$T/n = 2/7 = 0.286 < 1/3$ 。 $t < n/3$ の側なので、ブロードキャストは 1 対 1 通信から作れる (§6 で効く)。

分割は体の上ではなく整数上

値 v (幅 β) を

$$v = \sum_{p=1}^n x_p \quad (\text{整数として})$$

と分け、各 x_p は $[0, 2^{\beta+\sigma})$ 一様、 $\sigma = 40$ 。

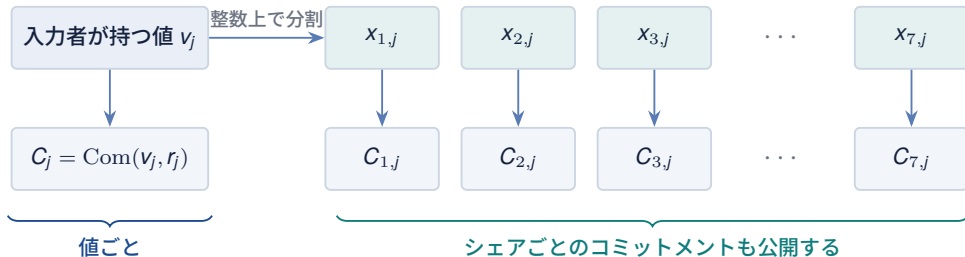
秘匿は完全でなく統計的に $2^{-\sigma}$ 。

なぜそうしたか：配る時点でエンジンの法を決めたくなかったため。

この選択が §3.7 まで尾を引く。

出所・注: security.tex § 設定。 $T < n/3$ の含意は Goyal–Song–Zhu (CRYPTO 2020)。

何がコミットされるか



シェアごとのコミットメント $C_{p,j}$ は、もともと (1) ノードが自分の受け取ったものを検算できるように、
(2) 誰でも $\prod_p C_{p,j} = C_j$ を確かめられるように置いてある。

§3.5 では、これがそのまま「誰が入れ替えたか」を名指す材料になる。— 追加で公開するものは無い。

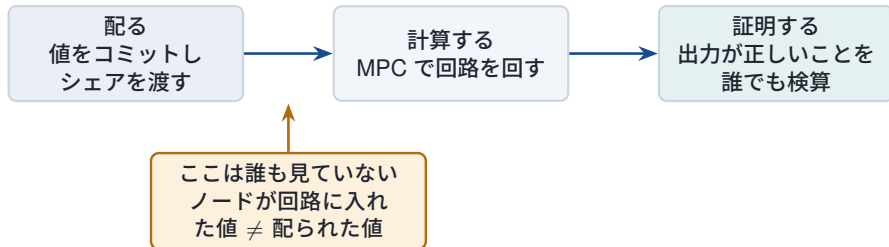
§2

継ぎ目

公開検証可能 MPC は、入力の同一性までは言っていない

BDO14 の構図と、その外側

Baum–Damgård–Orlandi 2014 が与えるもの：出力の公開検証可能性



ノードは、自分のシェアをコミットメントと照合したうえで、別の数に入れられる

MPC は「入力として与えられた値」に対して正しく計算する。与えられた値が正しいかは、MPC の保証の外にあるし、MP-SPDZ のトランスクリプトにそのずれは現れない — この継ぎ目を塞ぐのが §3。

§3

入力整合検査

1つの線形結合を開けるだけ。ただし、開ける順番が全て

構成

回路は、実際に食わされた値 v'_j とマスク m' から

$$s = \sum_{j=1}^m c_j v'_j + m'$$

を計算し、ブラインドとともに開示する。検証者は公開されたコミットメントだけを見て

$$\text{Com}(s, \beta) \stackrel{?}{=} C_m \cdot \prod_j C_j^{c_j}$$

を確かめる。

公開係数 × 秘密値は局所演算なので、結合そのものは通信ゼロ。代金は開示 1 回だけ。

出所・注: security.tex § 健全性。マスク m は §5 の補題で効く。

効いている性質

Pedersen が加法準同型なので、検証者は v_j を 1 つも知らないまま右辺を組み立てられる。

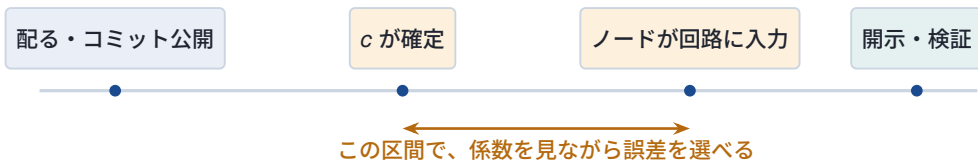
残る問い (次ページ)

c_j は、いつ決まるのか？

最初の版はコミットメントから導いていた。「値 → コミットメント → 係数」の順だから後だ、という理屈で。

その理屈は間違っていた

コミットメントは配る時点で公開され、ノードが回路に食わせるのはその後——つまりノードは、自分の入力を決める前に c を読める。



命題 1 (零空間攻撃)

$m \geq 2$ とする。ノードは $v_1 + c_2k$ と $v_2 - c_1k$ を入れる。すると $\sum_j c_j e_j = c_1 c_2 k - c_2 c_1 k = 0$ が恒等的に成り立つ。

当て推量は無く、上げるべき安全性パラメータも無い—— c の核は次元 $m - 1$ あり、そこに入る誤差はすべて通る。

出所・注: 実装で確認済み——`verify_per_party` は初回の試行で「不正者なし」を返した (`artifacts/coefficient_timing_flaw.json`)。

なぜ 40 本のテストが見つけれなかったか

入力を 1 つだけ差し替えた場合

条件は $c_1 e_1 = 0$ 。 $c_1 \neq 0$ だから $e_1 = 0$ が強制される。

つまり単独差し替えは、どんな係数でも本当に捕まる。

テスト 40 本、成果物 6 件、エンドツーエンドの名指し実行 — 全部通った。

攻撃には 2 つ要る

そしてテストは全部、1 つしか差し替えていなかった。

欠陥は量子子の中にあった——「ある入力について成り立つ」と「すべての誤差ベクトルについて成り立つ」の差。

量子子は、テストスイートが確かめないものの筆頭である。

これが「証明を書く」ことの実利である。実装は動き、テストは緑で、成果物は再現できた。それでも構成は健全ではなかった。見つけたのは、定理の仮定を書き下そうとして「 c が敵の選択と独立」がどこにも保証されていないと気づいたときである。

訂正：チャレンジは入力後に引く

定理 1 健全性

敵が ρ の開示より前に $(v', m') \neq (v, m)$ を固定したとする。検証者が受理する確率は高々

$$2^{-\lambda+1} + \text{Adv}^{\text{bind}}.$$

証明の骨：受理すれば、コミットメントの 2 通りの開示を作った (Adv^{bind}) か、 $\sum_j c_j e_j + e_m = 0$ が成り立つ。ある $e_{j^*} \neq 0$ を取り他を条件付ければ、式は c_{j^*} を一意に定める。 ρ を後に引いたことで c_{j^*} は敵の選択と独立に一樣である。

$c_j = \rho^j$ を MPC の法 p で取ると、もっと綺麗になる。 $\sum_k \rho^k e_k + e_m = 0$ は ρ の m 次多項式なので、根は高々 m 個——Schwartz–Zippel で m/p 。

166 値・253 ビット体で約 2^{-245} 。置き換える前は 6 ビット係数 7 回で 2^{-42} だった。

直したことを、直ったと言う前に確かめる

仕掛けた攻撃	結果
2 入力に $+\delta / -\delta$ を置いて相殺を狙う	名指し
チャレンジを推測し、その零空間から誤差を選ぶ	名指し
誤差を固定し、新しいチャレンジを 20,000 回引く	生き残り 0 回

エンドツーエンド (host-a、7 ノード実機)	結果
正直な実行	検証成功・誰も名指されず
ノード 4 がテイカーの数量を差し替え	ノード 4 を名指し

「正直な実行で誰も名指されない」を先に確かめる理由。以前これを飛ばして**正直な 7 ノードの実行が全員を有罪にした** — コミットメントは 253 ビット体、回路は既定の 128 ビットで、開示が巻いていた。偽陽性は、帰責の仕組みが取りうる最悪の壊れ方である。見逃しより悪い。

出所・注: artifacts/sound_check.json, artifacts/identity.json。

誰がやったかまで言う：ノードごとの検査

定理 2 名指し

定理 1 の仮定を各計算者に適用して：(i) P_p が配られたものと違うものを入れれば、確率 $1 - (2^{-\lambda+1} + \text{Adv}^{\text{bind}})$ 以上で p が名指される。(ii) 正直な P_p は決して名指されない（完全 — 検査が恒等式になるため）。(iii) 同時に差し替えるノードが何台でも成り立つ。

(iii) が §4 の復号による名指しとの決定的な差である。復号は「訂正能力」で頭打ちになるが、こちらは各ノードが自分のコミットメントに対して独立に立っているので上限が無い — 5 台が嘘をついた場合こそ、運用者は名前を要る。

そして各 s_p を足すと元の集約検査になるので、定理 2 は定理 1 を含む。

値段

host-a, $n=7$, 253 ビット体, 166 値	ラウンド	全体通信
検査なし	70	83.0829 MB
集約 (不健全)	71	83.0986 MB
ノードごと (健全)	72	83.4257 MB +1 ラウンド、+0.39%

3 つとも平文の参照解と照合済み。

当たった予測

+1 ラウンド。 ρ の開示が 1 回増え、検査の開示ど
うしは独立なので同じラウンドに乗る。

外した予測

通信は下がると書いた。開示が 49 回 → 8 回に減
るため。実際は集約比で上がった。

64 ビット乱数の生成が前処理のランダムビットを
食うためと思うが、切り分けていないので推測で
ある。

出所・注: artifacts/sound_check.json。予測は challenge_fix_prediction.json に測る前から置いてある。

法 p にしたら、幅の予算が丸ごと消えた

消えたもの (BINDING.md §3.1)

- 必要幅 164 ビットの計算
- マスクがスラックを二度払う話
- 係数を狭くする取引と、その 2^{-34} の天井

全部「検査を整数上で取っていて、どちらの体でも巻けない」からだけ存在していた。

法 p なら

両辺とも $\text{mod } p$ の主張なので、避けるものが無い。マスクは**一様な体要素 1 つ**になる。

代わりに要求するのはコミットメントの体と MPC の体を揃えること。

体を揃える理由は、これで独立に 3 つ目になった。(1) 閾値パラメータが素直に組み立つ。(2) ずれていると監査が**全ノードを誤って有罪にする**。(3) 検査が綺麗なのは法 p のときだけ。

代金は測ってある：通信 2.00×、ラウンド 1.00×。壁時計は 15 ms で 1.07×、120 ms で 1.06× — **遠いほど安い**。

出所・注: artifacts/matched_field.json。

§4

もう一つの名指し

シェアは、すでに誤り訂正符号である

復号は「検出」でなく「訂正」ができる

定理 3 一意な名指し

次数 d の Shamir シェア n 個は Reed–Solomon 符号 $RS[n, d + 1]$ 、最小距離 $n - d$ をなす。Berlekamp–Welch は、誤りが $\lfloor (n - d - 1)/2 \rfloor$ 個以下なら秘密と、誤った位置の集合を厳密に復元し、それを超えるとどんな算法も不可能。

	次数 d	訂正能力	
通常値	2	2	$= T$ 。耐える不正はすべて名指せる
次数低減前の積	4	1	$= T - 1$ 。ここに隙間がある

ただし開示は $2t + 1 = 5$ シェアしか集めていない。 $RS[5, 3]$ は距離 3 で訂正は 1 個。 $T = 2$ の訂正には 7 つ全部が要る — 開示通信 40% 増、実測で本計算 1.50×、全体 1.33×、ラウンドは不変。名指しは「た」ではないが、ラウンドは 1 つも増えない。

出所・注: patches/locate-inconsistent-shares.patch、実測は artifacts/decode_patch.json。

トランスクリプトは、誰が不正したか言うか

host-a、あるノードの前処理ファイルを 1 バイト壊し、-F で実行、毎回復元。

壊したノード	トランスクリプトに出た文字列
3	Fatal error at fp2000-0:3 (MULS): inconsistent Shamir secret sharing
1	完全に同一 — この 3 は命令のオフセットであって、ノード番号ではない
5	Fatal error in communication: read_some: stream truncated
6	Fatal error in communication: read_some: stream truncated

最初の 2 行で決着する：違う犯人に同じ文字列が出るので、トランスクリプトはノードの情報を運んでいない。

後の 2 行はもっと悪い：メッセージが指しているのは**壊れた相手ではなく、接続が切れた側**である。——「エラーに番号が出ているから帰責できている」は、読めば違うと分かる。

出所・注: artifacts/locate.json。

§5

秘匿性

開示された 1 本の式から、方針は復元できるか

マスクが引き受けているもの

補題 開示は値を隠す

$W = \sum_j c_j v_j$ 、 $|W| < 2^\omega$ とし、マスク m を $[0, 2^{\omega+\sigma})$ 上一様とする。 $s = W + m$ と $[0, 2^{\omega+\sigma})$ 上の一様分布との統計距離は高々 $2^{-\sigma}$ 。

証明： s は $[W, W + 2^{\omega+\sigma})$ 上一様で、これは $[0, 2^{\omega+\sigma})$ と高々 $|W| < 2^\omega$ 点しか食い違わない。よって距離は $2^\omega / 2^{\omega+\sigma} = 2^{-\sigma}$ 。

マスクが無ければ、開示は方針についての線形方程式 1 本である。見積もりを繰り返し取れば連立して解ける — 検査そのものが、探りの道具になる。

マスクはそれを塞ぐが、マスク自身も入力なので、スラックを持って配られる。そしてそのスラックが、§3.7 で消した「幅の予算」の正体だった。

整数版での必要幅：集約 164 ビット、ノードごと 160 ビット（マスクが自分の入力で配られないため）。出荷されている素数は 127 ビット — なお法 p 版では、この計算自体が要らなくなる。

§6

壊れたときに何が起きるか

中断・名指し・そして「止まらない」は、本当に可能なのか

段は2つではなく5つある

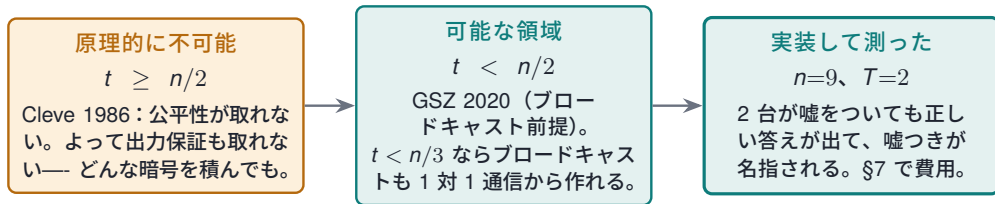
	何と言えるか	要るもの	出典
①	出力は正しいか、出力が無い	過半数が不正でも可	—
②	... 正直者は誰がかも知る	過半数が不正でも可	IOZ 2014
③	... トランスクリプトを読む第三者も知る	掲示板	—
④	... 裁定者がトランスクリプトだけで判定し、中断を説明するのに足る人数を名指す	掲示板	Küsters 2010
⑤	中断が起きない。不正者が何をしても正直者は出力を得る	honest majority	GSZ 2020

公開検証可能性は、この段の上に無い。直交する軸である——「答えが正しく、外部が検算できる」。それが BDO 2014 で、この実装が持っているもの。

公開検証可能でありながら ① で止まることは矛盾なく起きる。そして、それがここの現状である。

⑤ 「壊れても動く」は可能か

可能。ただし「壊れた数が閾値以下なら」という条件付きで、そこには理論的な壁がある。



そして実際の線は $t < n/3$ ではなく $n \geq 4t + 1$ だった。次数 $2t$ の積がそのまま訂正できるのがこの線で、区切りも脱落者も王も要らなくなる。 $T=2$ なら $n=9$ 。

そして無条件版は存在しない。3/7 が結託したら、どんな構成も止まるか間違うかしかない——「壊れても動く」は常に「閾値以下なら動く」である。

「壊れる」の中身で難易度が3段違う

壊れ方	何で足りるか	ここでの状態
落ちる（クラッシュ・回線断）	冗長性だけ。次数 2 を 7 点に配ってあるので 3 点あれば復元できる	復元は可能。ただしプロトコルを続けるのは別問題
でたらめを送る	局所復号。Berlekamp–Welch で訂正と名指しが同時に出る	実装済み（能力 2、開示 1.33×、ラウンド不変）
整合しているが違うものを送る（乗算の途中で）	ここが本番。ただし $n \geq 4t + 1$ なら王を消して全員で復号すれば足りる	実装済み（ $n=9$ 、2 台まで訂正・名指し・止まらない）

そして商品としての「止まらない」は、MPC だけでは決まらない。メイカーが見積もりを出していなければ、GSZ でも注文は作れない。

GSZ が買うのは「入力が確定した後、ノードが結果を拒否できない」ことである。中断は選択的失敗攻撃の面になる — 気に入らないノードが落とせばやり直しが起き、その回数そのものが情報になる。そこを閉じる。

§7

頑健性の値段

測ってみたら、費用ではなく節約だった

勾配で測る：1 乗算あたりの 1 者あたり体要素数

総量の比ではなく勾配：SIMD 1 行で回路サイズを 2 つ取り、通信差 ÷ 乗算差。乗算を加算に置換した対照回路の勾配は 0 — 入力共有と 1 回の開示は勾配に入っていない。

host-a, $n=7$, $T=2$, 128 ビット体	要素/者/乗算	ラウンド
semi-honest Shamir (単一相)	5.714	4 (一定)
malicious Shamir ・ 本計算のみ	8.000	3 (一定)
malicious Shamir ・ 単一相 (実行中に三つ組生成)	48.231	17 → 137
GSZ 2020 ・ 出力保証つき ・ 単一相	5.5 – 7.5	—

出力保証は、いま動いている本計算だけよりも安い。全体比では **6.4**× 安い。しかも与えるものは strictly 多い — **止まらないのだから**。

GSZ は単一相で情報理論的：前処理を臨界経路から外す余地が無いので、5.5 が全部である。

出所・注: `artifacts/multiplication_cost.json`。

測定器を、信じる前に較正した

対照になっている理由

ハーネスは別の問いのために作った。その semi-honest 側は、合わせに行っていない数字に対する対照になる。

GSZ は「 $t < n/2$ で最良の semi-honest は 5.5」
と書いている。ハーネスは **5.714** を返した — 論文の数字に対して 4%。

この測定が繰り返しかけた誤り 2 つ

(1) 最初の版は 22,000 乗算に **22,002** ラウンドを測った — 逐次の `for_range` で全部が臨界経路に乗っていた。直すと 2,000 から 220,000 乗算まで 3 ラウンド一定。

(2) 前処理を単一相で測って本計算の費用と呼ぶ誤りは **edaBits** で一度やって **2** 桁外している。だから 48.2 と 8.0 を別々に残す。

GSZ との比較には総量を使う (GSZ に前処理相が無い)。ただし本計算の **8.000** が、実際の前処理を持つ配備が払う額である。 — 混ぜれば、都合のよい側が有利になる。

「SPDZ に GSZ を」が違う3つの理由

(1) 文として成り立たない SPDZ は過半数不正前提：全シェアに MAC、 $n - 1$ 台まで耐える。GSZ は正直過半数前提：Shamir、情報理論的、保証は $t < n/2$ だから存在する。片方に片方を入れるのは統合ではない — SPDZ の閾値では、その保証がそもそも無い。

なお、この実装は SPDZ を使っていない。同じリポジトリに同梱されている Shamir 系を使っている。

(2) 正しく言い直しても、差し替え可能な部品ではない MP-SPDZ の口は「固定した参加者集合に対する `init_mul` / `prepare_mul` / `exchange` / `finalize_mul`」。GSZ が要るのは回路を区切って検査点を置くこと、脱落に応じて参加者と閾値が途中で変わること（=再共有）、王と中継の非対称性。— 仮想機械の変更であって、既存クラスの隣に置く新クラスではない。

(3) 入れても帰責は手に入らない GSZ の特定は他の参加者に向いている。掲示板の読者に向いていない — §6 の ⑤ は取れるが ④ は取れない。別軸である。

王を消す — e は秘密ではない

ATLAS の王が弱点 王は $2t + 1$ 点から補間して配り直す。嘘つきの王は、整合した、値の違う共有を配れる — 正しい符号語をいくら訂正しても捕まらない。

「整合しているが違うものを送る」は、ここに集まっている。

だから王が要らない r は毎回新しい次数 $2t$ の乱数なので、 $e = xy + r$ は秘密ではない。

全員に開けば各自が Berlekamp–Welch で訂正し、 $[xy]_t = e - [r]_t$ を局所に計算する。言葉を取る相手が消える。

GSZ に要ると書いた 3 つ

王と中継

二重共有 $\{[r]_{2t}, [r]_t\}$

区切り・検査点・脱落者の再共有

エンジンを読んだ結果

もう入っている (`base_king`, `next_king`)

もう入っている (`get_double_sharing`)

要らなくなった (王を消したので)

出所・注: `patches/robust-atlas.patch`、`--options robust`。乱数生成も `get_hyper` で hyper-invertible 行列を既に使っている。

実測：止まらなかった

host-a、 $n=9$ 、 $T=2$ 、2,000 乗算。名指した者が全ての覆われた積で嘘をつく。

嘘つき	答え	名指し	
なし	正しい	—	8 ラウンド
{0}	正しい	[0]	止まらない
{0, 1}	正しい	[0, 1]	止まらない
{8}	正しい	[8]	止まらない
{0, 1, 2}	—	—	能力超過と言って拒む
$n = 7$	—	—	起動を拒む（能力 1、閾値 2）

	要素/者/乗算	ラウンド
ATLAS（王あり、準正直） $n=9$	5.333	9 → 13
これ（王なし、訂正あり） $n=9$	18.222	8 → 12
悪意 Shamir + 名指し $n=7$ （配備中）	64.236	11 → 23

配備中のものに対して **1 者あたり 0.28×**、**全体 0.37×**、ラウンドも少ない。そして止まらない。

出所・注: artifacts/robust_atlas.json。予測 11.3 は **1.6 倍外した**（実測 18.222） — 送信側だけを数えたモデルで、切り分けていない。

頑健なのは「どこまで」か

頑健になったもの 乗算の次数低減の一段だけ。
ここが、嘘つきを名指しても**プロトコルが止まっていた**場所である。

なっていないもの 二重共有 ($[r]_t$ と $[r]_{2t}$ を互いに不整合にされうる)、出力の開示、そして入力相。

二重共有は前処理なので、中断してよい。入力を持たないので漏れるものが無く、誰の結果も拒否できない。やり直せばよく、失敗し続けるノードは取引と取引の間に外す — 統治の動作である。

入力相は QOMM ではもう閉じている（この試験台では閉じていない）。ディーラーがシェアごとのコミットメントを公開しているので、悪いシェアを渡されたノードは「開かない」ことを誰にでも示せる — 頑健な入力相が要する苦情プロトコルは、公開検証可能性の代金で払ってある。

費用は帯域ではなく **2 社増えること**。POSITION.md 自身が「一番の risk は連合が組めないこと」と書いている。

$n=7$ のまま $T=1$ でも条件は満たすが、**2 社の結託で板が全部読める**ので venue には取れない。

§8

VOLE-in-the-Head

実装して測ったら、113× の優位が消えた

何を作ったか

検証者が Δ を選び、証明者が $\{t_x\}$ を持つとき、 $q = \sum_{x \neq \Delta} t_x(\Delta - x) = \Delta u + v$ が全ての Δ について同時に成り立つ。証明者が頭の中で全ての Δ を走らせ、最後に 1 つだけ開けるのが「in the head」。

道具立ては GGM 木の 1 つを除く全部開示、 $[\tau, 1, \tau]$ の繰り返し符号、部分空間 VOLE。FAEST と同じ骨格。

狙い

ハッシュしか使わない—— 離散対数を仮定しない。したがって耐量子。

いまの Pedersen は、量子計算機の下で束縛性が消える。

測る前に書いた予測：

算術演算だけを数えると 113× 有利（楕円曲線のスカラー倍が要らないため）。

比較の相手を揃えるため、両者を同じ口に嵌めた—— `LinearProofScheme (prove_linear / verify_linear / proof_bytes)`。Pedersen と VOLEitH が同じ検査に対して差し替え可能になる。

測った結果：予測は方向ごと外れていた

入力数	証明	検証	バイト数
16	15.01×	22.52×	11.45×
64	7.51×	13.26×	8.98×
167（実配備の規模）	5.88×	11.16×	8.39×

いずれも **VOLEitH** が **Pedersen** より「悪い」倍率。113× 有利のはずだった。

証明サイズの内訳（167 入力）：**VOLE** 整合補正 **40,080** バイト — 全体の **88%**、木の開示は **4%**。

理由は一行で書ける。FAEST が住んでいる $\mathbb{F}_2$ では、整合補正はビットである。127 ビット素体では **16 バイトの体要素**になる。

113× は **Pedersen** に対する優位ではなく、「離散対数を仮定しないこと」の値段だった。 — 証人がビットでなく体要素だと、公開検証可能性への変換で消える。

出所・注: artifacts/voleith.json。同じ Rust・同じ機械。Pedersen 側だけ高度に最適化済み。16 入力の時間は再実行で 1 割動くが、バイト数とハッシュ数は不変。

木を深くすると縮むが、ハッシュが指数で増える

深さ	葉/木	繰り返し	バイト	ハッシュ
4	16	32	89,136	1,024
8	256	16	45,616	8,192
12	4,096	11	32,080	90,112
16	65,536	8	23,856	1,048,576

いずれも健全性 128 ビット以上。深さ 4→16 でサイズは 3.7× 縮み、ハッシュは 1024× 増える。

線形符号は差し替えでは済まない。一般符号が持つのは k_C 個ずつの塊をまたぐ線形結合だけで、ここで証明したい文は値ごとに係数が違う内積、つまり塊の中の結合である。塊の中を取り戻すには論文の Π_{2D-LC} (図 6) が要り、部分空間 VOLE を $2l+2$ 行呼び $(l+1) \times n_C$ 行列を開く。

符号の置換だけを数えると 10,816 バイト ([31, 16, 16]) だが、図 6 の送信まで数えると 18,896 バイト ([47, 32, 16])。深さ 8 の 45,616 バイトに対して 2.4× であって 4.2× ではない。

どちらも算術だけで、実装していない。

出所・注: artifacts/voleith.json の sweep と linear_code。

§8.5

どの台帳に載せたか

Rust 製 Avalanche VM で、予約から決済まで一つの状態遷移にした

決済 VM は、Rust で作った Avalanche L1 として動く

AvalancheGo が合意形成とネットワークを担い、QOMM の Rust VM が取引の検査と正本台帳の更新を担う。汎用契約の上に載せるのではなく、QOMM 専用の状態遷移を RPCChainVM protocol 45 で接続した。

5

バリデータ

1.14.2

AvalancheGo

Rust

QOMM の状態遷移

受入試験で確認したこと。5 台の論理バリデータが同じ VM ハッシュを読み込み、取引を確定し、再起動後に同じ状態へ復帰した。高速な状態同期も完了した。

検証環境の限界。5 バリデータと 7 つの MPC 処理は、現時点では 1 台の物理ホスト上で動かしている。複数拠点での実測ではない。

出所・注: `artifacts/avalanche_l1_acceptance.json`、`artifacts/avalanche_qomm_full_acceptance.json`。

口座を指定せず、正本台帳をどう更新するのか

DeFMI（分散型金融市場基盤）は、名前付き口座の残高表ではなく、**一回限りの電子証書（ノート）**の集合を正本として持つ。

台帳が公開して持つもの	証書の要約値、使用済み印、状態の要約値
決済で消費するもの	Maker と Taker が見積もり前に確保した、一回限りの予約証書
決済で作るもの	受取人だけが見つけて使える、新しい請求権の証書
公開しないもの	当事者名、元口座、先口座、秘密鍵、予約の内訳

受入試験では元口座・先口座の使用数がともに 0 だった。CSD 発行者の署名は外部コマンドへ委ね、DeFMI は秘密鍵を読み込まない。ただし、実機の鍵管理装置を使ったことまでは**未確認**である。

出所・注: `artifacts/avalanche_gomm_full_acceptance.json` の `account_free_notes` と `csd_issuer`。

決済への同意は、見積もりの後ではなく前に取る

見積もりを見た後に署名を求めると、断って探る余地が残る。そこで Maker と Taker の双方が、**問い合わせ前に決済可能額を予約する。**

Maker	定期的に、最大在庫または保証枠を予約してから価格規則を提供する
Taker	資金またはネット保証枠を予約し、その署名を添えて問い合わせる
7 つの MPC 処理	注文と価格規則を秘密のまま照合し、3 者以上で予約用のゼロ知識支払指図 (zkPI) へ署名する
DeFMI	zkPI と予約証書を検査し、見積もり後の追加署名なしで決済する

受入試験では、鍵の持分を一か所へ集めず、分離した 7 処理のうち 3 処理で署名した。価格計算の故障許容数 $T=2$ と、支払指図の署名条件 3/7 は別の条件である。

出所・注: `artifacts/avalanche_qomm_full_acceptance.json` の `pretrade_authorization`。

同時 RFQ は、受付順と一つの状態根で枠超過を止める

RFQ（価格照会）が同時に来ても、別々の残枠を見て両方を通してはいけない。Rust VM は署名付き受付番号の順に、同じ正本状態へ逐次適用する。

2 RFQ

一つの原子的取引

53

確定した高さ

0

競合失敗時の変更

各約定は、法人・保証提供者・資産ごとの残枠を同じ状態から差し引く。CCP 保証、銀行保証、自己保証を同じ型で扱う。古い残枠を前提にした二重使用は取引全体を拒否し、途中の更新を残さない。

実測では受付番号 1 と 3 の 2 件を、5 バリデータが 1 取引として高さ 53 で確定した。

出所・注: `artifacts/avalanche_qomm_full_acceptance.json` の `settled_rfqs` と `concurrency`。

§9

証明していないこと

何を主張してよく、何を主張してはいけないか

証明していないことの一覧

プロトコル本体	MP-SPDZ の malicious honest-majority Shamir は安全と仮定して引用している。こちらのパッチは失敗時に何を報告するかを変えたのであって、保証を変えていない
合成	UC の扱いは無い。部品ごとに証明し、合成は議論で繋いでいる
集約検査	訂正はライブラリと回路のノードごとモードに入っている。集約モードは、いまも入力の前に係数を固定したまま出力される — 費用の比較対象として残しており、生成器はそう書いたプログラムを吐き、標準エラーにも出す
頑健性と中断	名指しは防止ではない。止める者を名指すのに $1.33\times$ 、入力を差し替える者を名指すのに $+1$ ラウンド $\cdot +0.39\%$ 。どちらも止めることはできない
入力者そのもの	守る気の無い方針にコミットするディーラーは、この枠組みの外側にある。コミットメントは値を束縛するのであって、意図を束縛しない

使っている暗号技術は、すべて既存のものである。新しいのは (1) それらを市場の形に繋いだこと、(2) その値段を測ったこと、(3) 決済まで中身を開かずに通したこと、(4) 繋ぎ目の健全性の欠陥を直したこと。

公開しているもの

github.com/shukob/qomm

github.com/shukob/zkpi

github.com/shukob/defmi

本体。回路生成器・MP-SPDZ 連携・監査

証明の側。コミットメント、入力整合検査、VOLE-in-the-Head

決済の側。口座名を出さない受け渡し (DvP) と Rust 製 Avalanche VM

3 つは独立した公開リポジトリである。英語の README・研究資料に加え、日本語の実装解説と本スライド 3 種を公開している。ホスト名は `host-a`~`host-d` に置き換えてある。

このスライドの数値はすべて `artifacts/*.json` に対応する。測る前に予測を `*_prediction.json` として置き、測ってから照合している。

外れた予測も同じ場所に残す — 何を外したかが読めない測定は、当たった分も確かめようがない。

まとめ

1. 公開検証可能 **MPC** の継ぎ目は、入力の同一性である。出力の正しさは外から検算できるが、ノードが何を入れたかはその外側にある。
2. そこを塞ぐ検査には、順番という仮定が隠れていた。係数をコミットメントから導くと、ノードは核から誤差を選べる — 恒等的に、確率 1 で。
3. 見つけたのは、テストではなく証明である。40 本のテストは全部、入力を 1 つしか差し替えていなかった。攻撃には 2 つ要る。
4. 直した後の値段は +1 ラウンド、+0.39%。そして幅の予算が丸ごと消えた。
5. 「壊れても動く」は、**GSZ** を書かずに手に入った。効く線は $t < n/3$ でなく $n \geq 4t + 1$ で、 $n=9$ なら王を消して全員で復号すれば足りる。2 台が嘘をついても答えは正しく、嘘つきは名指され、止まらない — 配備中の 0.28×。
6. **VOLEitH** の 113× は、公開検証可能性への変換で消えた。証人がビットでなく体要素だと、整合補正が証明の 88%を占める。